

Task Line - Project Architecture & Concepts Guide

This document outlines the core concepts, data flows, plugins, and architectural decisions used in building the **Task Line** application. It is designed as a study guide to help you prepare for technical interviews.

1. Core Concepts Used

React Hooks

- **useState**: Used extensively for local component state (e.g., managing form inputs, UI toggles like modal visibility or dropdowns, and temporary expanding/collapsing states).
- **useEffect**: Used for side effects such as syncing the dark/light theme to localStorage, fetching data when the component mounts, listening to Firebase Auth state changes, and auto-resizing textareas dynamically.
- **useRef**: Used for direct DOM manipulation without triggering re-renders. Examples include focusing the search input automatically and detecting "clicks outside" a modal or dropdown to close it.
- **useContext (via Custom Hook useDataHook)**: Prevents prop-drilling by providing a global state object to all deeply nested components.

Optimistic UI Updates

When a user adds a task, the UI immediately reflects the new task by generating a temporary ID and pushing it to the local state. The application then makes a background API call to the backend. Once the real database object is returned, the temporary task is seamlessly swapped with the real database object. This creates a hyper-fast, lag-free user experience.

Modern CSS & Tailwind Features

- **Glassmorphism**: Achieved using bg-white/20 (background opacity) combined with backdrop-blur-md. This creates a frosted glass effect that blends dynamically with the application's underlying gradient.
 - **Dark Mode Strategy**: Implemented via Tailwind's dark: variant and class strategy. The app explicitly adds or removes the dark class on the <html> root element based on the themeMode state, which allows for instant manual toggling.
 - **CSS Grid & Flexbox**: Used for fully responsive layouts. The layout morphs from stacked columns on mobile to multi-column dashboards on desktop effortlessly.
-

2. Data Flow Architecture

The application follows a centralized state management pattern using the **Context API**.

1. State Initialization (App.jsx):

- Core states (tasks, user profile, theme) are declared at the root level in App.jsx.
- API call functions (addTask, updateTask, deleteTask) are also defined here.

2. Context Provider:

- App.jsx wraps the entire application with <Provider value={{...}}>.

3. Component Consumption:

- Deeply nested components (like NewTasks.jsx or Header.jsx) import the custom hook `const { updateTask } = useDataHook();` to dispatch actions or read state directly, skipping intermediate components.

Authentication Data Flow

1. User logs in via Firebase Auth.
 2. `onAuthStateChanged` (in `App.jsx`) detects the login event and retrieves the user object.
 3. The app requests a Secure JWT Token via `user.getIdToken()`.
 4. This token is passed in the Authorization header to the custom Node/Express backend (`axios.get(..., { headers: { Authorization: Bearer ${token} } })`) to securely fetch the user's specific tasks from MongoDB.
-

3. Libraries & Plugins Used

- **Vite:** Ultra-fast frontend build tool and development server, replacing Create React App (CRA).
 - **React.js:** The core UI library.
 - **Tailwind CSS v4:** Utility-first CSS framework used for all styling without writing custom CSS files.
 - **Firebase Auth:** Provides secure, battle-tested user authentication without managing passwords manually.
 - **Axios:** Promise-based HTTP client used to interface with the backend REST API.
 - **React DOM Portal (`createPortal`):** Used to render the `ProfileSettingsModal` outside the normal DOM hierarchy, ensuring it always floats perfectly on top without CSS z-index conflicts.
-

4. Interview Questions & Answers

Q1: Why did you choose the Context API over Redux for this project?

Answer: "For Task Line, the global state primarily consisted of the user session, the current theme, and a single list of tasks. Redux requires significant boilerplate (actions, reducers, store configuration) which would have been over-engineering for a project of this scale. Context API, combined with React Hooks, provided a lightweight and perfectly native solution to avoid prop-drilling without adding extra bundle size."

Q2: How did you implement the Glassmorphism UI?

Answer: "I utilized Tailwind CSS. To achieve the glass effect, you need two properties: a semi-transparent background color (like `bg-white/20`) and a backdrop filter to blur whatever is behind it (`backdrop-blur-md`). This combination creates the frosted glass look. I also added subtle borders (`border-white/40`) to mimic the reflective edge of glass."

Q3: Explain how your Optimistic UI update works when adding a task.

Answer: "When a user adds a task, waiting for the server response can feel sluggish. Instead, my `addTask` function creates a temporary task object with `Date.now()` as a fake ID and immediately updates the React state. The UI instantly shows the task. In the background, it fires the Axios POST request. When the server responds with the actual MongoDB document, I update the state again, silently swapping the temporary task with the real one."

Q4: How does your application handle Theme persistence (Light/Dark mode)?

Answer: "I initialized the theme state by lazily reading from the browser's localStorage inside the useState hook. I then set up a useEffect that listens for changes to the themeMode state. Whenever the user toggles the theme, the useEffect updates the <html> class list to trigger Tailwind's dark mode, and simultaneously saves the new preference to localStorage. This ensures the theme persists across page reloads and browser sessions."

Q5: How did you fix the layout issues when a task description was incredibly long?

Answer: "Initially, long tasks caused the fixed-height container to hide text, and allowing it to grow ruined the vertical centering of the action icons. I solved this by removing the fixed height and switching the flexbox alignment from items-center to items-start, while adding calculated top padding (pt-7). This anchored the action icons perfectly to the top line of text regardless of how tall the text wrapper grew. I also added an interactive expand/collapse toggle using line-clamp."